

ابن تطبيقك الأول مع Claude Code

تطبيق هاتف عربي حقيقي من الفكرة إلى ملف التثبيت: الواجهة والحالة والتخزين والواجهات والأذونات والاختبارات والبناء

01	الفكرة والمتطلبات وقصص المستخدم
02	مقارنة أطر التطبيقات واختيار واحد
03	الواجهة والحالة والتخزين والواجهات
04	الأذونات والأمان والاختبارات والبناء

محدث في سبتمبر 2026

10 مراحل

تطبيق كامل

برومبتات عربية

جميع الحقوق محفوظة لقناة برومبتات عربية.

هذا المنتج مخصص للاستخدام الشخصي للمشتري. لا يجوز إعادة نشره أو بيعه أو توزيعه أو اقتباس أجزاء منه لأغراض تجارية دون إذن كتابي من المالك.

القناة الرسمية: <https://t.me/PromptsArabic>

اسم المنتج	ابن تطبيقك الأول مع Claude Code
الإصدار	الإصدار الأول
اللغة	العربية
المقاس	A4 عمودي — مهيأ للقراءة على الهاتف
تمت مراجعة معلومات وميزات 6 سبتمبر 2026 هذا الدليل حتى تاريخ	
مصادر التحقق	docs.flutter.dev و reactnative.dev و learn.microsoft.com/dotnet و /code.claude.com/docs

ملاحظة مهمة عن تاريخ المحتوى

منصات الذكاء الاصطناعي تتغير بسرعة. كل ميزة مذكورة هنا تم التحقق منها من المصدر الرسمي بتاريخ 6 سبتمبر 2026. إذا مر وقت طويل على هذا التاريخ، راجع الصفحة الرسمية للميزة قبل الاعتماد عليها في عمل حساس.

عن هذا الدليل

تطبيق الهاتف ليس موقعا بشاشة أصغر. له دورة حياة مختلفة، وتخزين محلي، وأذونات، وبناء ينتهي بملف تثبيت. هذا الدليل يبني معك تطبيق متابعة مصروفات عربيا كاملا على عشر مراحل، ولكل مرحلة برومبت فعلي وملفات متوقعة واختبار يدوي ومعياري انتقال. ويبدأ قبل ذلك بمقارنة أطر العمل، فتعرف لماذا اخترنا ما اخترناه ومتى يكون غيره أنسب.

لمن هذا الدليل

- من يعرف أساسيات البرمجة ولم يبن تطبيق هاتف من قبل
- طلاب يحتاجون تطبيقا حقيقيا لمشروع تخرج
- مطورو ويب ينتقلون إلى تطبيقات الهاتف
- من يريد تطبيقا عربيا صحيح الاتجاه لا تطبيقا مترجما

ماذا ستتعلم

- مقارنة عملية بين Flutter و React Native و NET MAUI، وقرار مبني على معايير
- بناء واجهة عربية RTL صحيحة على مستوى التطبيق لا على مستوى النص
- إدارة الحالة بطريقة لا تتحول إلى فوضى بعد ثلاث شاشات
- التخزين المحلي عبر طبقة مستودع تسمح بتغيير المصدر لاحقا
- استدعاء واجهة خارجية مع معالجة كل حالات الفشل
- الأذونات على أندرويد ومتى تطلب وكيف تتعامل مع الرفض
- الاختبارات والأمان وبناء ملف تثبيت جاهز للنشر

قبل أن تبدأ

المتطلبات

- جهاز يستطيع تشغيل محاكي أندرويد، أو هاتف أندرويد وكابل
- Claude Code مثبت ويعمل
- معرفة أساسية بالبرمجة الكائنية
- مساحة قرص كافية – أدوات تطوير الهاتف كبيرة
- صبر في مرحلة الإعداد: هي أطول مرحلة وأقلها متعة

كيف تستخدم هذا الدليل

- لا تخطط مرحلة الإعداد ولا تستعجلها. أكثر ما يوقف المبتدئين هو بيئة نصف جاهزة
- أرسل برومبت كل مرحلة كما هو بعد تعبئة المتغيرات
- اختبر على جهاز حقيقي لا على المحاكي وحده، خصوصا الأذونات والأداء
- بعد كل مرحلة: راجع الفروق ثم اختبر ثم commit
- احتفظ بملف القرارات وحديثه بعد كل مرحلة

طريقة القراءة الموصى بها

لا تقرأ الدليل كله دفعة واحدة. اقرأ فصلا واحدا، ثم نفذ تمرينه على مهمة حقيقية من عملك، ثم انتقل. أربعة فصول مطبقة أفضل من أربعة عشر فصلا مقروءا.

7	● الفكرة والنطاق وقصص المستخدم
7	التطبيق الذي سنبنيه
7	قصص المستخدم
7	خارج النطاق صراحة
8	برومبت تحديد النطاق
9	● مقارنة الأطر واختيار واحد
9	المرشحون وحالتهم وقت المراجعة
10	المقارنة على معايير مشروعنا
10	القرار
12	● البيئة وإعداد المشروع
14	ضبط الاتجاه العربي على مستوى التطبيق
15	● الشاشات والتنقل
17	● إدارة الحالة
17	ثلاث طبقات لا تختلط
20	● التخزين المحلي الدائم
21	شكل واجهة المستودع
22	● النماذج والتحقق ومعالجة الأخطاء
24	● الأذونات والتذكير اليومي
24	قواعد طلب الإذن

26	● الواجهة الخارجية والاختبارات
26	استدعاء واجهة خارجية
27	الاختبارات
28	● الأداء وتجربة الاستخدام
28	الأداء
28	تجربة الاستخدام
30	● الأمان والبناء وقائمة الإصدار
30	الأمان قبل البناء
30	البناء
31	التوقيع
31	قائمة تحقق الإصدار
32	● ورقة المراجعة السريعة
32	المراحل ومعايير الانتقال
33	أوامر ستستخدمها يوميا
33	دورة العمل في كل مرحلة
33	أخطاء شائعة في أول تطبيق
34	ما تضعه في CLAUDE.md لهذا المشروع

الفكرة والنطاق وقصص المستخدم

التطبيق الأول يفشل غالبا لأنه واسع لا لأنه صعب. هذه المرحلة تضيقه حتى يصبح قابلا للإنهاء.

التطبيق الذي سنبنيه

مصرفي: تطبيق يسجل فيه المستخدم مصروفاته اليومية، ويصنفها، ويرى ملخصا شهريا لكل تصنيف، ويستقبل تذكيرا يوميا. اخترناه لأنه يمر بكل ما يحتاجه أي تطبيق حقيقي: نموذج إدخال، وقائمة، وتصفية، وتخزين محلي دائم، وحساب وعرض، وإذن نظام، وبناء ونشر — بلا تعقيد يشنت.

قصص المستخدم

1. كمستخدم أريد تسجيل مصروف بسرعة حتى لا أنسى تسجيله.
2. كمستخدم أريد تصنيف كل مصروف حتى أعرف أين يذهب مالي.
3. كمستخدم أريد رؤية ملخص الشهر حتى أقارن تصنيفاتي.
4. كمستخدم أريد أن تبقى بياناتي بعد إغلاق التطبيق حتى لا أعيد الإدخال.
5. كمستخدم أريد تذكيرا يوميا حتى أبني عادة التسجيل.

خارج النطاق صراحة

المستبعد	سبب الاستبعاد
حسابات ومزامنة سحابية	تضيف خادما ومصادقة إلى تطبيق أول
ربط بحسابات بنكية	تعقيد قانوني وأمني لا يناسب مشروعا تعليميا
رسوم بيانية متقدمة	الملخص الرقمي يكفي للقيمة الأساسية
دعم iOS في هذا الدليل	بناء iOS يتطلب جهاز macOS وحساب مطور
تعدد العملات	يضاعف منطق الحساب دون قيمة في النسخة الأولى

لماذا أندرويد فقط

الأطر التي سنقارنها تدعم iOS، لكن بناء ملف iOS يتطلب جهاز macOS وحساب مطور مدفوع. اخترنا مسار أندرويد حتى يستطيع كل قارئ إنهاء المشروع فعلا. الكود نفسه سيعمل على iOS عند توفر الأدوات.

برومبت تحديد النطاق

أرسل هذا قبل إنشاء أي مشروع

سنبني تطبيق هاتف. لا تكتب أي كود في هذه المرحلة.

الفكرة:

[الفكرة]

المستخدم المستهدف:

[المستخدم]

ما يفعله اليوم بدون التطبيق:

[البديل الحالي]

المدة المتاحة:

[المدة]

وقي الأسبوعي:

[الوقت الأسبوعي]

المنصة المستهدفة: أندرويد

أنشئ REQUIREMENTS.md يحتوي:

أولا. المشكلة في ثلاث جمل

ثانيا. قصص المستخدم الأساسية، خمس على الأكثر

ثالثا. الشاشات المطلوبة، وما يفعله المستخدم في كل شاشة

رابعا. البيانات: ما الحقول التي نخزنها، ونوع كل حقل، وأيها إلزامي

خامسا. داخل النطاق، وخارج النطاق مع سبب كل استبعاد

سادسا. معايير القبول: متى نعتبر التطبيق منتهيا

سابعا. ما الذي يحتاج إذن نظام، ولماذا

قواعد:

التطبيق عربي واتجاهه من اليمين إلى اليسار.

إذا كانت الفكرة أوسع من مدتي، قل ذلك واقترح تضيقها.

لا تفترض معلومة عني، أسأل.

مقارنة الأطر واختيار واحد

ثلاثة خيارات جادة، ولكل واحد حالة يكون فيها الأفضل. القرار الصحيح هو الذي تعرف سببه، لا الذي يختاره الأكثر.

المرشحون وحالتهم وقت المراجعة

الإطار	اللغة	الحالة وقت المراجعة
Flutter	Dart	الإصدار المستقر 3.47
React	JavaScript أو	الإصدار 0.85 شحن ضمن حزمة Expo SDK 56 في مايو 2026، والإصدارات
Native	TypeScript	شهرية
NET MAUI.	#C	على NET 10. وهو إصدار مدعوم طويل الأمد في الدعم النشط

تحقق قبل أن تبدأ

دورة إصدار React Native شهرية، وأطر التطبيقات تتحرك بسرعة. الأرقام أعلاه تم التحقق منها بتاريخ 6 سبتمبر 2026. افتح صفحة الإصدارات الرسمية للإطار الذي تختاره وتأكد من أحدث إصدار مستقر قبل أن تبدأ.

المقارنة على معايير مشروعنا

المعيار	Flutter	React Native	NET MAUI.
سهولة التعلم من الصفر	متوسطة: لغة جديدة	عالية لمن يعرف JavaScript	عالية لمن يعرف #C
كل شيء في لغة واحدة	نعم	نعم	نعم
الحاجة إلى كود أصلي للأساسيات	قليلة	متوسطة	قليلة
سهولة بناء ملف أندرويد	عالية	متوسطة	متوسطة
أدوات تشخيص البيئة	ممتازة	متوسطة	جيدة
دعم RTL جاهز	نعم	نعم	نعم
حجم الأمثلة العربية	كبير	كبير	أصغر
ملاءمة العمل مع أداة وكيلة	عالية: مشروع بلغة واحدة وبنية واضحة	متوسطة: إعدادات أكثر	عالية

القرار

نختار **Flutter** لهذا الدليل، لثلاثة أسباب تخص القارئ المبتدئ:

1. بيئة التطوير تشخص بأمر واحد، وهذه أكبر عقبة أمام المبتدئ في تطوير الهاتف.
2. بناء ملف أندرويد خطوة واحدة تقريبا، بلا سلسلة أدوات متفرقة.
3. المشروع بلغة واحدة وبنية ملفات واضحة، فتقرؤه أنت وتقرؤه الأداة الوكيلية بسهولة.

ومتى يكون غيره أنسب؟

- **React Native** إذا كنت مطور ويب بالفعل وتريد إعادة استخدام معرفتك بـ JavaScript وحزمها.
- **NET MAUI.** إذا كان فريقك يعمل بـ #C ولديكم خادم .NET تريدون مشاركة نماذج البيانات معه. وهذا يخص قارئ نظم المعلومات الذي يبني الخلفية بـ ASP.NET Core.

القرار الذي يهم أكثر من الإطار

أيا كان الإطار الذي تختاره، افصل منطق التطبيق عن الواجهة. الكود الذي يحسب ويخزن ويتحقق يجب ألا يعرف شيئاً عن الشاشة. هذا القرار هو الذي يحدد إن كان تطبيقك قابلاً للاختبار والنقل، لا اسم الإطار.

البيئة وإعداد المشروع

أطول مرحلة وأقلها متعة. أنجزها كاملة الآن ولن تعود إليها.

ما الذي نبنيه في هذه المرحلة

بيئة تطوير تعمل، ومشروع Flutter يفتح على محاكي أو جهاز، ومستودع Git، وملف CLAUDE.md يعرف فيه Claude Code أوامر مشروعك.

ما ترسله إلى Claude Code

أرسل هذا كما هو بعد تعبئة المتغيرات

جهاز مشروع Flutter جديدا للتطبيق.

المطلوب:

أولا. تحقق أولا من بيئتي وأخبرني بما ينقصها قبل أي إنشاء

ثانيا. أنشئ مشروعا باسم [اسم المشروع] بمعرف حزمة

[معرف الحزمة]

ثالثا. اضبط اللغة العربية والاتجاه من اليمين إلى اليسار على مستوى التطبيق كله

رابعا. أنشئ بنية مجلدات: طبقة نماذج، وطبقة بيانات، وطبقة منطق، وطبقة واجهة

خامسا. أنشئ .gitignore مناسباً لمشروع Flutter

سادسا. أنشئ CLAUDE.md فيه أوامر التشغيل والاختبار والفحص والبناء، وقرار أن التطبيق عربي RTL

قواعد:

استخدم أحدث إصدار مستقر من الإطار، ولا تستخدم قناة beta أو dev.

اعرض الخطة قبل التنفيذ وانتظر موافقتي.

لا تضيف حزمة خارجية في هذه المرحلة إلا إذا كانت ضرورية، واذكر سبب كل حزمة.

الملفات المتوقعة إنشاؤها أو تعديلها

- ملف إعدادات الحزم والاعتماديات
- نقطة الدخول الرئيسية للتطبيق
- بنية المجلدات: models و data و logic و ui
- .gitignore و CLAUDE.md

كيف تختبر بنفسك

1. شغل أمر تشخيص البيئة وتأكد أنه بلا أخطاء حمراء.
2. شغل التطبيق على محاكي وتأكد أنه يفتح.
3. شغله على جهاز حقيقي عبر الكابل وتأكد أنه يفتح أيضا.
4. غير نسا وتأكد أن إعادة التحميل السريعة تعمل.
5. شغل `git status` وتأكد أن مجلدات البناء غير مدرجة.

أخطاء محتملة

- تخطي تشخيص البيئة والمضي رغم وجود تحذيرات — ستعود لاحقا في أسوأ وقت.
- معرف حزمة مؤقت مثل `com.example`، وتغييره بعد النشر مستحيل عمليا.
- إضافة عشر حزم في اليوم الأول قبل أن تعرف أيها تحتاج فعلا.

معييار الانتقال

التطبيق يفتح على جهاز حقيقي، والاتجاه من اليمين لليسا، والمستودع نظيف.

```
import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';

void main() ⇒ runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      locale: const Locale('ar'),
      supportedLocales: const [Locale('ar'), Locale('en')],
      localizationsDelegates: const [
        GlobalMaterialLocalizations.delegate,
        GlobalWidgetsLocalizations.delegate,
        GlobalCupertinoLocalizations.delegate,
      ],
      home: const HomeScreen(),
    );
  }
}
```

الاتجاه ليس محاذاة نص

ضبط اللغة على مستوى التطبيق يقلب التخطيط كله: الحشوات والهوامش وترتيب العناصر واتجاه أيقونات الرجوع. أما محاذاة النص إلى اليمين في كل شاشة على حدة فهي إصلاح شكلي يترك بقية الواجهة معكوسة. واستخدم الحشوات الموجهة بالبداية والنهاية بدل اليسار واليمين.

الشاشات والتنقل

الهيكل الظاهر أولاً: شاشات حقيقية وتنقل يعمل، بيانات وهمية.

ما الذي نبنيه في هذه المرحلة

ثلاث شاشات: القائمة، وإضافة مصروف، والملخص؛ مع تنقل يعمل وحالاتها الأربع.

ما ترسله إلى Claude Code

أرسل هذا كما هو بعد تعبئة المتغيرات

ابن شاشات التطبيق ببيانات وهمية.

الشاشات:

شاشة القائمة: قائمة المصروفات مرتبة من الأحدث، وزر إضافة عائم

شاشة الإضافة: نموذج إدخال مصروف

شاشة الملخص: إجمالي الشهر وإجمالي كل تصنيف

لكل شاشة اعرض الحالات الأربع:

تحميل، وفارغة، وخطأ، وطبيعية

التنقل:

تنقل واضح بين الشاشات، مع زر رجوع يعمل بشكل صحيح في واجهة من اليمين لليسا

الحفاظ على حالة الشاشة عند الرجوع إليها

قواعد:

استخدم بيانات وهمية ثابتة الآن، وسنبذل المصدر في المرحلة التالية.

لا منطق حساب داخل شيفرة الواجهة.

كل نص ظاهر بالعربية.

اعرض الخطة قبل التنفيذ.

الملفات المتوقعة إنشاؤها أو تعديلها

lib/ui/screens/ — ثلاث شاشات •

lib/ui/widgets/ — بطاقة مصروف، وحالة فارغة، وحالة خطأ، ومؤشر تحميل •

lib/models/expense.dart •

كيف تختبر بنفسك

1. تنقل بين الشاشات الثلاث ذهابا وإيابا.
2. تأكد أن سهم الرجوع يشير في الاتجاه الصحيح للواجهة العربية.
3. أفرغ البيانات الوهمية وتأكد من ظهور الحالة الفارغة لا شاشة بيضاء.
4. دور الجهاز وتأكد أن التخطيط لا ينكسر.
5. جرب على شاشة صغيرة وشاشة كبيرة.

أخطاء محتملة

- منطق الحساب داخل شيفرة الواجهة، فيصبح غير قابل للاختبار.
- بناء الحالة الطبيعية فقط، فتظهر شاشة بيضاء عند أول قائمة فارغة.
- نصوص إنجليزية مؤقتة تبقى إلى النثر.

معييار الانتقال

الشاشات الثلاث تعمل بحالاتها الأربع، والتنقل صحيح في الاتجاه العربي.

إدارة الحالة

إدارة الحالة هي المكان الذي يتحول فيه التطبيق البسيط إلى فوضى. القاعدة: مصدر واحد للحقيقة، والواجهة تعرضه ولا تملكه.

ثلاث طبقات لا تختلط

الطبقة	مسؤوليتها	ما لا تعرفه
الواجهة	عرض الحالة والاستجابة للمس	من أين جاءت البيانات
المنطق	الحساب والتحقق وقواعد العمل	شكل الشاشة
البيانات	القراءة والكتابة من المصدر	لماذا طلبت البيانات

ما الذي نبنيه في هذه المرحلة

طبقة حالة واحدة تملك قائمة المصروفات وتوفر عمليات الإضافة والحذف والتصفية والحساب، وواجهة تستمع إليها فقط.

أرسل هذا كما هو بعد تعبئة المتغيرات

أضف إدارة حالة للتطبيق.

المطلوب:

مصدر واحد للحقيقة يملك قائمة المصروفات
عمليات: إضافة، وحذف، وتصفية بالتصنيف، وتصفية بالشهر
قيم محسوبة: إجمالي الشهر، وإجمالي كل تصنيف
حالة التحميل وحالة الخطأ ضمن الحالة نفسها لا منفصلة في كل شاشة

قواعد:

استخدم حلاً مناسباً لحجم هذا التطبيق، ولا تدخل إطار إدارة حالة ثقيلًا بلا حاجة.
اشرح لي سبب اختيارك قبل التنفيذ.
منطق الحساب في طبقة المنطق لا في الواجهة، وقابل للاختبار بلا واجهة.
الواجهة تستمع للحالة ولا تعدلها إلا عبر العمليات المعروفة.

اعرض التصميم قبل التنفيذ وانتظر موافقتي.

الملفات المتوقعة إنشاؤها أو تعديلها

• `lib/logic/expenses_state.dart`

• `lib/logic/summary.dart` — دوال الحساب الصافية

• تعديل الشاشات الثلاث لتستمع للحالة

كيف تختبر بنفسك

1. أضف مصروفاً من شاشة الإضافة وتأكد أنه يظهر في القائمة فوراً.
2. احذف مصروفاً وتأكد أن الملخص يتحدث فوراً.
3. صف بتصنيف وتأكد أن الملخص يعكس التصفية أو لا يعكسها حسب ما قررت.
4. ابحث في شيفرة الشاشات عن أي عملية حسابية — يجب ألا توجد.

أخطاء محتملة

- نسخة من البيانات في كل شاشة، فتتناقض الشاشات.
- حساب الإجمالي داخل بناء الواجهة، فيعاد حسابه في كل إطار.

- إدخال إطار إدارة حالة ضخم لتطبيق بثلاث شاشات.

معييار الانتقال

تستطيع اختبار كل دوال الحساب بلا تشغيل واجهة.

التخزين المحلي الدائم

التطبيق الذي يفقد بياناته عند الإغلاق ليس تطبيقًا. هذه المرحلة تجعله حقيقيًا.

ما الذي نبنيه في هذه المرحلة

مستودع بيانات بواجهة واضحة، وتنفيذ يحفظ في تخزين محلي دائم، مع ترحيل البيانات الوهمية إليه.

ما ترسله إلى Claude Code

أرسل هذا كما هو بعد تعبئة المتغيرات

أضف تخزينًا محليًا دائمًا.

صمم أولًا واجهة مستودع فيها:

جلب كل المصروفات

جلب مصروفات شهر معين

إضافة مصروف

حذف مصروف بمعرفه

ثم نفذ هذه الواجهة بتخزين محلي دائم.

قواعد:

طبقة المنطق تتعامل مع الواجهة فقط ولا تعرف نوع التخزين.

اختر حزمة تخزين مستقرة ومصانة، وتحقق من إصدارها قبل الإضافة، وشرح لي سبب اختيارها.

عالج حالة أول تشغيل حيث لا توجد بيانات.

عالج حالة تلف البيانات المخزنة بلا إسقاط التطبيق.

لا تخزن أي بيانات حساسة في تخزين غير مشفر.

اعرض تصميم الواجهة قبل التنفيذ.

الملفات المتوقعة إنشاؤها أو تعديلها

• lib/data/expense_repository.dart – الواجهة المجردة

• lib/data/local_expense_repository.dart – التنفيذ

• تعديل طبقة الحالة لتستخدم المستودع

• تحديث ملف الاعتماديات

كيف تختبر بنفسك

1. أضف مصروفين، ثم أغلق التطبيق كلياً، ثم افتحه: هل بقيا؟
2. احذف مصروفاً ثم أعد التشغيل: هل بقي محذوفاً؟
3. امسح بيانات التطبيق من إعدادات النظام ثم افتحه: هل يفتح بلا انهيبار؟
4. أضف خمسين مصروفاً وراقب سرعة فتح القائمة.

أخطاء محتملة

- استدعاء التخزين مباشرة من الشاشة، فتفقد فائدة المستودع.
- افتراض أن البيانات المخزنة سليمة دائماً.
- عمليات تخزين متزامنة على الخيط الرئيسي، فتتجمد الواجهة.

معييار الانتقال

البيانات تبقى بعد إغلاق التطبيق، وتغيير نوع التخزين يتم في ملف واحد.

شكل واجهة المستودع

lib/data/expense_repository.dart

```
abstract class ExpenseRepository {  
  Future<List<Expense>> getAll();  
  Future<List<Expense>> getByMonth(int year, int month);  
  Future<void> add(Expense expense);  
  Future<void> deleteById(String id);  
}
```

النماذج والتحقق ومعالجة الأخطاء

كل مدخل من المستخدم هو فرصة لخطأ. النموذج الجيد يمنع الخطأ ويشرح سببه.

ما الذي نبنيه في هذه المرحلة

نموذج إضافة مصروف بتحقيق كامل، ورسائل خطأ عربية بجانب كل حقل، ومعالجة أخطاء موحدة على مستوى التطبيق.

ما ترسله إلى Claude Code

أرسل هذا كما هو بعد تعبئة المتغيرات

أكمل نموذج إضافة المصروف وتحققه.

الحقول: المبلغ، والتصنيف، والتاريخ، وملاحظة اختيارية.

قواعد التحقق:

المبلغ رقم موجب أكبر من صفر وأقل من حد أعلى معقول
المبلغ يقبل الفاصلة العشرية ويرفض الحروف
التصنيف مختار من قائمة محددة لا نص حر
التاريخ لا يكون في المستقبل
الملاحظة لا تتجاوز مئتي حرف

السلوك:

رسالة خطأ عربية بجانب الحقل المسؤول لا رسالة عامة
زر الحفظ معطل أثناء الحفظ لمنع الإضافة المزدوجة
عند النجاح: إغلاق الشاشة ورسالة تأكيد قصيرة
عند الفشل: رسالة واضحة وإبقاء ما أدخله المستخدم

معالجة الأخطاء:

معالج موحد يحول أخطاء الطبقات الدنيا إلى رسائل مفهومة
لا تعرض تفاصيل تقنية للمستخدم، وسجلها بدل ذلك

اعرض قواعد التحقق كدوال قابلة للاختبار بلا واجهة.

الملفات المتوقعة إنشاؤها أو تعديلها

• `lib/logic/validation.dart` — دوال تحقق صافية

• شاشة الإضافة ومكوناتها

كيف تختبر بنفسك

1. احفظ نموذجاً فارغاً وتأكد من ظهور رسائل عربية بجانب كل حقل.
2. أدخل مبلغاً سالباً، ثم صفراً، ثم نصاً، ثم رقماً ضخماً جداً.
3. اختر تاريخاً في المستقبل وتأكد من الرفض.
4. اضغط حفظ مرتين بسرعة وتأكد من عدم إضافة سجلين.
5. أدخل ملاحظة بمئتي حرف وواحد وتأكد من الرفض.

أخطاء محتملة

- تحقق داخل شيفرة الواجهة فقط، فلا يمكن اختباره.
- رسالة خطأ واحدة عامة لا تدل على الحقل.
- فقدان ما أدخله المستخدم عند فشل الحفظ.

معييار الانتقال

كل قواعد التحقق مغطاة باختبارات تعمل بلا واجهة.

الأذونات والتذكير اليومي

الإذن ليس خانة تعلم. هو عقد مع المستخدم: تطلبه في وقته، وتشرح سببه، وتعمل بشكل معقول إن رفضه.

قواعد طلب الإذن

1. لا تطلب إذنًا عند فتح التطبيق أول مرة. اطلبه عند أول استخدام فعلي للميزة.
2. اشرح للمستخدم لماذا تحتاجه قبل أن تظهر نافذة النظام.
3. إذا رفض، لا تعطل التطبيق كله. عطل الميزة وحدها واترك له طريقة لتفعيلها لاحقًا.
4. إذا رفض نهائياً، وجهه إلى إعدادات النظام بدل تكرار الطلب.

ما الذي نبنيه في هذه المرحلة

تذكير يومي بإشعار محلي، مع طلب إذن الإشعارات في وقته الصحيح، ومسار كامل لحالة الرفض.

ما ترسله إلى Claude Code

أرسل هذا كما هو بعد تعبئة المتغيرات

أضف تذكيراً يومياً بإشعار محلي.

السلوك:

شاشة إعدادات بسيطة يفعل فيها المستخدم التذكير ويختار وقته عند التفعيل لأول مرة: اشرح سبب الإذن ثم اطلبه عند الرفض: أوقف التذكير، واعرض رسالة تشرح كيف يفعله من إعدادات النظام لاحقاً عند القبول: جدول إشعارات يومياً في الوقت المختار الإشعار بالعربية ونصه واضح

قواعد:

اطلب الإذن عند تفعيل الميزة لا عند فتح التطبيق تعامل مع اختلاف سلوك الأذونات بين إصدارات أندرويد تحقق من حالة الإذن عند كل فتح، فقد يسحبه المستخدم من الإعدادات اذكر بالضبط ما الأذونات التي ستضاف إلى ملف البيان ولماذا اعرض الخطة قبل التنفيذ، وأخبرني بأي إذن إضافي يتطلبه الحل قبل إضافته.

الملفات المتوقعة إنشاؤها أو تعديلها

- `lib/logic/reminder_service.dart`

- شاشة الإعدادات

- ملف بيان أندرويد – الأذونات

- تحديث ملف الاعتماديات

كيف تختبر بنفسك

1. فعل التذكير وارفض الإذن: هل يبقى التطبيق يعمل ويشرح ما حدث؟
2. فعله واقبل الإذن، واضبط الوقت بعد دقيقتين، وانتظر الإشعار.
3. اسحب الإذن من إعدادات النظام ثم افتح التطبيق: هل يكتشف ذلك؟
4. أعد تشغيل الجهاز وتأكد أن التذكير ما زال مجدولا.
5. اختبر على إصدار أندرويد حديث وإصدار أقدم إن أمكن.

أخطاء محتملة

- طلب كل الأذونات عند الإقلاع، وهو أسرع سبب لحذف التطبيق.
- افتراض أن الإذن الممنوح يبقى ممنوحا إلى الأبد.
- إضافة إذن لا تحتاجه لأن حزمة ما طلبته – راجع ملف البيان بنفسك.

معييار الانتقال

التذكير يصل فعلا، ورفض الإذن لا يكسر التطبيق.

الواجهة الخارجية والاختبارات

استدعاء واجهة خارجية

سنضيف ميزة اختيارية: تحويل الإجمالي إلى عملة أخرى عبر واجهة خارجية. الهدف التعليمي ليس التحويل، بل التعامل مع شبكة تفشل.

برومبت الواجهة الخارجية

أضف ميزة تحويل الإجمالي إلى عملة أخرى عبر واجهة خارجية.

المطلوب:

طبقة خدمة منفصلة تتولى الاستدعاء، ولا تستدعيها الواجهة مباشرة

مهلة زمنية لكل طلب

معالجة منفصلة لكل حالة: انقطاع الشبكة، ومهلة منتهية، ورد بحالة خطأ، ورد بصيغة غير متوقعة

تخزين مؤقت لآخر سعر ناجح، مع عرض وقت آخر تحديث

إذا فشل الطلب: اعرض آخر سعر محفوظ مع تنبيه أنه قديم، ولا تعرض خطأ فقط

قواعد أمنية:

إذا كانت الواجهة تتطلب مفتاحاً فلا تضعه في الكود ولا في ملف يصل إلى المستودع

اشرح لي أين يجب أن يوضع المفتاح ولماذا وضعه في التطبيق نفسه ليس آمناً

اعرض التصميم قبل التنفيذ.

حقيقة عن الأسرار في تطبيقات الهاتف

أي مفتاح تضعه داخل تطبيق مثبت على جهاز المستخدم يمكن استخراجه. تشفيره يزيد الصعوبة ولا يمنع. القاعدة: المفاتيح الحساسة تبقى على خادم تملكه، والتطبيق يستدعي خادمك لا الخدمة مباشرة.

برومبت الاختبارات

أضف اختبارات لهذا التطبيق.

الأولوية بهذا الترتيب:

- أولا. دوال الحساب: الإجمالي، وإجمالي كل تصنيف، والتصنيفية بالشهر
- ثانيا. قواعد التحقق: كل قاعدة وكل حالة حدية
- ثالثا. المستودع: الإضافة والحذف وال جلب، بتخزين وهمي في الذاكرة
- رابعا. اختبار واجهة واحد على الأقل: إضافة مصروف تظهر في القائمة

لكل اختبار:

- اسم يصف السلوك المتوقع
- اختبار واحد لسلوك واحد
- بلا اعتماد على شبكة ولا على تخزين حقيقي على القرص

ثم:

- اذكر ما لم تختبره ولماذا
- اذكر أي جزء صعب اختباره وما التغيير في التصميم الذي يسهله
- أضف أمر تشغيل الاختبارات إلى CLAUDE.md

نوع الاختبار	ما يغطيه	سرعته
اختبار وحدة	الحساب والتحقق والمنطق	سريع جدا
اختبار مكون واجهة	ظهور العناصر واستجابتها للمس	متوسط
اختبار تكامل	مسار المستخدم كاملا على جهاز	بطيء

أين تضع جهدك

غط المنطق والتحقق باختبارات وحدة كثيرة وسريعة، وأضف اختبار واجهة واحدا أو اثنين لأهم مسار، واترك اختبار التكامل لما قبل الإصدار. الهرم المقلوب الذي يعتمد على اختبارات التكامل وحدها بطيء وهش.

الأداء وتجربة الاستخدام

التطبيق البطيء يحذف. والتطبيق الذي لا يشرح نفسه لا يستخدم.

الأداء

- **القوائم الطويلة:** ابن عناصر القائمة عند الحاجة لا كلها دفعة واحدة.
- **العمليات الثقيلة:** لا تشغلها على الخيط الرئيسي، وإلا تجمدت الواجهة.
- **إعادة البناء:** لا تعد بناء شجرة الواجهة كلها عند تغير قيمة واحدة.
- **الصور:** حدد أبعادها ولا تحمل حجماً أكبر مما يعرض.
- **زمن الإقلاع:** لا تفعل عملاً ثقيلاً قبل أول إطار.

برومبت مراجعة الأداء

راجع أداء هذا التطبيق.

افحص:

القوائم: هل تبني عناصرها عند الحاجة أم كلها دفعة واحدة
إعادة البناء: أي جزء يعاد بناؤه بلا داع عند تغير الحالة
العمليات المتزامنة الثقيلة على الخيط الرئيسي
زمن الإقلاع: ما الذي يحدث قبل أول إطار
استهلاك الذاكرة عند خمسمئة سجل

لكل ملاحظة: الموضع، والأثر المتوقع، والإصلاح المقترح.
رتب من الأعلى أثراً. لا تعد كتابة الكود، اقترح فقط.

تجربة الاستخدام

1. الشاشة الأولى تشرح نفسها بلا جولة تعريفية طويلة.
2. أهم إجراء يبعد لمسة واحدة، لا ثلاثاً.
3. كل عملية تستغرق وقتاً لها مؤشر واضح.
4. كل خطأ يقول للمستخدم ماذا يفعل، لا ماذا حدث تقنياً.

5. الحالة الفارغة تشرح كيف يبدأ لا تكتفي بقول لا يوجد شيء.

6. لا حذف بلا تأكيد، ويفضل إتاحة التراجع.

اختبار الخمس دقائق

أعط التطبيق لشخص لم يره، واطلب منه تسجيل مصروف ورؤية ملخص الشهر، ثم اصمت تماما وراقب. كل مكان يتردد فيه هو عيب في التصميم لا في فهمه.

الأمان والبناء وقائمة الإصدار

الأمان قبل البناء

- لا مفتاح ولا رمز داخل الكود أو داخل الملفات التي تبني مع التطبيق.
- لا تسجل بيانات المستخدم في السجلات في نسخة الإصدار.
- راجع ملف بيان أندرويد: احذف أي إذن لا تحتاجه فعلا.
- لا تخزن بيانات حساسة في تخزين عادي غير مشفر.
- راجع الحزم الخارجية: كم منها تحتاجه فعلا، ومن يصونها.

برومبت المراجعة الأمنية قبل الإصدار

راجع هذا التطبيق أمنيًا قبل بناء نسخة الإصدار.

افحص:

أي سر أو مفتاح مكتوب في الكود أو في ملفات الإعداد التي تبني مع التطبيق
ملف بيان أندرويد: كل إذن مطلوب، وهل هو ضروري فعلا، ومن أضافه
السجلات: هل تسجل بيانات مستخدم في نسخة الإصدار
التخزين المحلي: هل فيه بيانات حساسة بلا حماية
الحزم الخارجية: أيها غير مصان أو له تحذيرات أمنية معروفة
الاتصال بالشبكة: هل كله مشفر

لكل ملاحظة: الخطورة، والموضع، وسيناريو الاستغلال، والإصلاح.
رتب من الأخطر. إذا كان الكود غير كاف للحكم، قل ذلك.

البناء

التحقق قبل البناء

```
flutter analyze
flutter test
flutter build appbundle --release
```

حزمة التطبيق أم ملف APK

حزمة التطبيق app bundle هي الصيغة التي يطلبها متجر Play، ويولد المتجر منها ملفات مخصصة لكل جهاز فتصغر نسخة المستخدم. أما ملف APK فمناسب للتوزيع المباشر والاختبار خارج المتجر. ابن الاثنين إن كنت تحتاج المسارين.

التوقيع

مفتاح التوقيع لا يفقد ولا يودع

مفتاح توقيع التطبيق هو هويته. إذا فقدته لن تستطيع نشر تحديث لتطبيقك على المتجر. احفظ نسخة منه في مكان آمن خارج جهازك. وفي الوقت نفسه: لا تودع المفتاح ولا كلمة مروره في المستودع أبدا. ضع بياناته في ملف مستبعد من Git، وأضف ذلك الملف إلى `.gitignore` قبل إنشائه.

قائمة تحقق الإصدار

1. التحليل الساكن بلا أخطاء، والاختبارات كلها تمر.
2. رقم الإصدار ورقم البناء محدثان.
3. اسم التطبيق وأيقونته وشاشة البدء نهائية لا افتراضية.
4. معرف الحزمة نهائي – لا يمكن تغييره بعد النشر.
5. لا نصوص إنجليزية مؤقتة ولا بيانات تجريبية.
6. الأذونات في ملف البيان هي ما تحتاجه فقط.
7. اختبر على جهازين مختلفين على الأقل بإصداري أندرويد مختلفين.
8. سياسة الخصوصية جاهزة إن كان التطبيق يجمع أي بيانات.
9. نسخة احتياطية من مفتاح التوقيع في مكان آمن.
10. خطة تراجع: ما الذي تفعله إن ظهر عطل حرج بعد النشر.

ورقة المراجعة السريعة

المراحل ومعايير الانتقال

المرحلة	معايير الانتقال
صفر – النطاق	خارج النطاق أطول من داخله، وقصص المستخدم خمس أو أقل
واحد – الإطار	قرار مكتوب بسببه، وإصدار مستقر متحقق منه
اثنان – البيئة	التطبيق يفتح على جهاز حقيقي، والاتجاه عربي
ثلاثة – الشاشات	الشاشات تعمل بحالاتها الأربع، والتنقل صحيح
أربعة – الحالة	دوال الحساب قابلة للاختبار بلا واجهة
خمسة – التخزين	البيانات تبقى بعد الإغلاق، والمصدر قابل للتبديل
ستة – النماذج	كل قواعد التحقق مغطاة باختبارات
سبعة – الأذونات	الرفض لا يكسر التطبيق، والتذكير يصل
ثمانية – الشبكة والاختبارات	فشل الشبكة يعرض آخر قيمة محفوظة لا شاشة خطأ
تسعة – الأداء والتجربة	شخص لم ير التطبيق، ينجز المهمة بلا شرح
عشرة – الإصدار	قائمة التحقق كاملة، والمفتاح محفوظ خارج المستودع

أوامر ستستخدمها يوميا

```
flutter doctor
flutter devices
flutter run
flutter analyze
flutter test
flutter build apk --release
flutter build appbundle --release
```

دورة العمل في كل مرحلة

```
git checkout -b phase/NN-name
claude --permission-mode plan
# review the plan, approve, then implement
/diff
flutter analyze && flutter test
git add -A && git commit
git switch main && git merge phase/NN-name
```

أخطاء شائعة في أول تطبيق

- البدء بالكود قبل تحديد النطاق وقصص المستخدم.
- منطق الحساب داخل شيفرة الواجهة، فلا يمكن اختباره.
- طلب كل الأذونات عند الإقلاع.
- افتراض أن الشبكة تعمل دائما.
- معرف حزمة افتراضي ينشر ثم يستحيل تغييره.
- مفتاح توقيع في المستودع، أو مفتاح ضائع بلا نسخة احتياطية.
- الاختبار على المحاكى وحده دون جهاز حقيقي.

ما تضعه في CLAUDE.md لهذا المشروع

CLAUDE .md

تطبيق مصروفي

أوامر

تشغيل: flutter run

تحليل: flutter analyze

اختبارات: flutter test

بناء إصدار: flutter build appbundle --release

قرارات

التطبيق عربي واتجاهه من اليمين إلى اليسار على مستوى التطبيق كله.

استخدم الحشوات الموجهة بالبداية والنهاية لا باليسار واليمين.

منطق الحساب والتحقق في طبقة المنطق، ولا يعرف شيئاً عن الواجهة.

الشاشات تتعامل مع المستودع عبر واجهته المجردة فقط.

لا مفاتيح ولا أسرار داخل التطبيق.

أعراف

كل نص ظاهر للمستخدم بالعربية.

كل دالة حساب لها اختبار وحدة.

الخلاصة والخطوة التالية

بنيت تطبيقًا حقيقيًا: بمتطلبات مكتوبة، وإطار اخترته بسبب، وطبقات لا تختلط، وتخزين دائم، وأذونات محترمة، واختبارات، وملف تثبيت موقع. والأهم أنك تعرف لماذا كل قرار.

ماذا تبني بعده

1. أضف تصديرا للبيانات إلى ملف، وستتعلم أذونات التخزين ومشاركة الملفات.
2. أضف مزامنة مع خادم، وستتعلم المصادقة وحل التعارضات.
3. انشر التطبيق فعلا ولو لدائرة صغيرة، فملاحظات مستخدم واحد حقيقي تساوي عشرة افتراضات.
4. أعد بناء الشاشات نفسها بإطار آخر لتعرف ما كان من الإطار وما كان منك.

بقية السلسلة

- Claude Code من الصفر إلى الاحتراف — لإتقان الأداة نفسها.
- ابن موقعك الأول مع Claude Code — الجانب الآخر من نفس المهارة.
- تعلم البرمجة مع Claude Code — إن شعرت أن الأساسيات ناقصة.

تنبيه عن الإصدارات

إصدارات Flutter و React Native و NET المذكورة تم التحقق منها من مصادرها الرسمية بتاريخ 6 سبتمبر 2026. أطر التطبيقات تتحرك بسرعة، ودورة إصدار React Native شهرية. تحقق من أحدث إصدار مستقر قبل أن تبدأ، ولا تبني على قناة beta أو dev.

برومبتات عربية

هذا المنتج من إنتاج قناة برومبتات عربية.
لمتابعة الأدلة والبرومبتات الجديدة:

<https://t.me/PromptsArabic>